

Software Architecture and Design Specification

Restaurant Management System

Document Information	Details
Document Type	Software Architecture and Design Specification
System	Restaurant Management System RMS
Version	1.0
Status	Design baseline for mini project implementation
Prepared for	Software Engineering Mini Project
Date	01 October 2026

Purpose

This specification defines a practical, secure, and implementable design for a single-outlet restaurant system that manages table bookings, menu items, orders, and live order tracking.

Document Control

Version	Date	Author	Change Summary
1.0	01 Oct 2026	Project Team	Initial architecture and design specification

Approval Record

Role	Name	Approval Status	Date
Faculty Evaluator	To be completed	Pending	To be completed
Project Team Representative	To be completed	Pending	To be completed

Table of Contents

Section	Title
1	Introduction
2	Architecture Overview
3	Component Architecture
4	Security Architecture
5	Detailed Design
6	Data Design
7	API and Interface Design
8	Error Handling and Validation
9	Deployment and Quality Attributes
10	Requirements Traceability and Verification
Appendix A	Glossary and Assumptions
Appendix B	Representative API Payloads

List of Figures

Figure	Title
--------	-------

Figure	Title
1	Restaurant Management System Component Architecture
2	Logical Data Model
3	Sequence Diagram Table Booking
4	Sequence Diagram Food Ordering and Live Tracking
5	Sequence Diagram Authentication and Authorization
6	Order Lifecycle and Allowed Transitions
7	Deployment View for a Single Restaurant Outlet

1 Introduction

1.1 Purpose

This Software Architecture and Design Specification SAD translates the approved Software Requirements Specification SRS for the Restaurant Management System RMS into an implementable solution. It defines the system structure, component responsibilities, data model, interfaces, security controls, failure behavior, and requirement traceability needed by the development team and evaluator.

1.2 Scope

RMS is a web application for one restaurant outlet. Customers can check table availability, reserve or cancel a table, browse available menu items, place an order, and follow its status. Staff can view and manage bookings and update orders. Admins can manage the menu and staff-facing operations. This version does not include payment-gateway integration, multiple branches, email, or SMS notifications.

1.3 Intended Readers

Reader	How this document is used
Developers	Implement the system consistently and make design decisions without redefining the approved requirements.
Faculty evaluators	Review architectural completeness, real-world feasibility, and traceability.
Restaurant staff and admin	Validate that daily booking, menu, and order operations are supported.

1.4 Design Objectives

- Prevent duplicate bookings even when two customers submit requests at nearly the same time.
- Ensure an order cannot be placed for an item that becomes unavailable after it was added to the cart.
- Persist the authoritative business state before a customer sees a confirmation or live update.
- Give each role only the permissions required for daily work.
- Keep the design simple enough to implement within a mini-project timeline while retaining clear extension points.

1.5 References

ID	Reference
R1	Software Requirements Specification Restaurant Management System Version 1.0, 06 September 2026.
R2	Test Plan Document Restaurant Management System, supplied project baseline.
R3	Software Engineering Mini Project Deliverables Part 1.
R4	Django and Django REST Framework documentation; Node.js and Socket.io documentation.

2 Architecture Overview

2.1 Architectural Pattern

RMS uses a layered web-application architecture with a separate real-time notification service. The presentation layer contains the customer and staff/admin web interfaces. Django REST Framework hosts the application layer, where authentication, validation, business rules, and database transactions are enforced. PostgreSQL is the system of record. A Node.js Socket.io service delivers persisted order-status changes to connected clients over authenticated WebSocket connections.

Layer	Primary Responsibility	Why It Is Needed
Presentation	Render responsive pages, collect input, show validation messages and real-time status.	Customers and restaurant staff need simple workflows on browser-based devices.
Application	Apply business rules, role checks, API validation and transactions.	Business decisions remain server-side and cannot be bypassed by changing the UI.
Real-time communication	Deliver only authorized order-status events.	A long-lived WebSocket avoids page refreshes and supports REQ-10.
Persistence	Store durable operational records and constraints.	Confirmed bookings and orders must survive server restarts.

2.2 Architecture Decisions

Decision	Rationale	Operational Consequence
Django REST Framework is the business API	Matches the SRS technology constraint and provides mature authentication, serializers, ORM, and security protections.	All state-changing REST requests pass through Django validation and transactions.
Node.js is limited to real-time delivery	Node.js is required by the SRS for WebSocket updates, but should not become a second owner of business rules.	Django persists status first and then publishes an internal event to Node.js.
PostgreSQL is preferred for production or demo	It supports transactions, constraints, row locking, and reliable concurrent access. SQLite remains acceptable only for a low-concurrency demonstration.	Database operations become the final protection against double booking.
Server derives availability from booking intervals	A permanent table Reserved flag cannot represent different time slots accurately.	A table may be available at 18:00 and booked at 19:00 on the same date.

2.3 Main Runtime Flow

A browser uses HTTPS REST requests for all reads and changes. The order-status tracking page also establishes a secure WebSocket connection. When staff update an order, Django validates the allowed transition, commits it to the database, and then sends an internal event to Node.js. Node.js publishes the event only to the authenticated room for that order. If an event is missed, the browser retrieves the current status from the REST API; the database remains authoritative.

3 Component Architecture

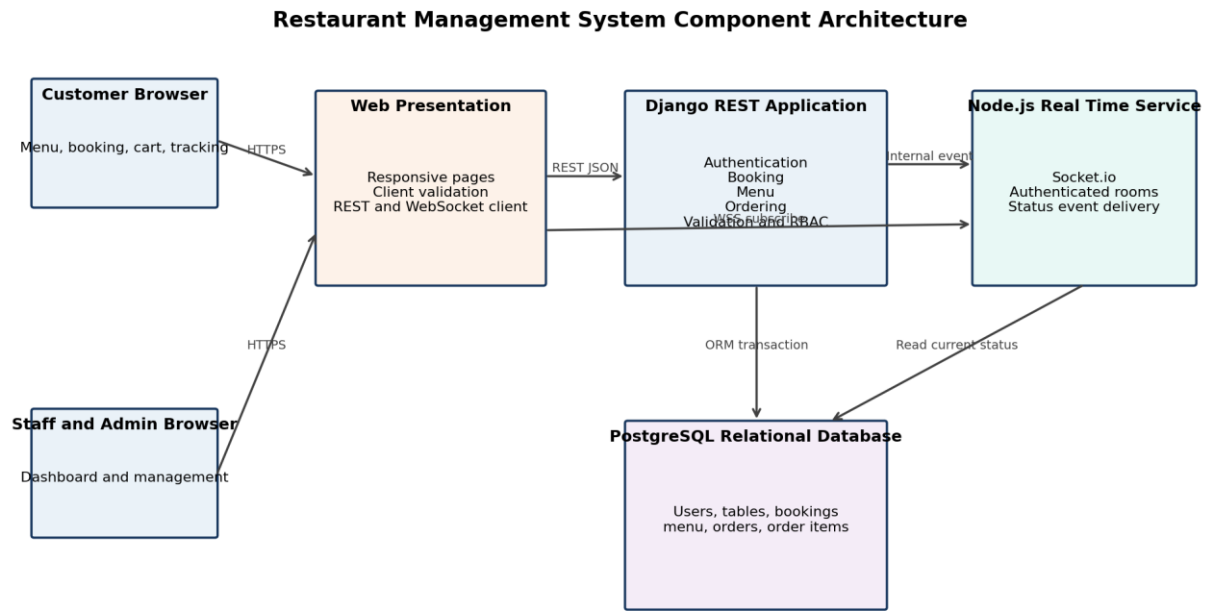


Figure 1. Restaurant Management System Component Architecture

3.1 Component Responsibilities

Component	Responsibilities	Provided Interfaces	Dependencies	Requirements
Customer Web UI	Browse menu, booking form, cart, tracking page, display messages.	HTTPS pages; REST client; WSS client.	Django API; Node.js service.	REQ-1, 2, 3, 6, 7, 8, 10, 11, 12
Staff and Admin UI	Booking search and override, order updates, menu management.	Protected HTTPS pages and REST client.	Django API.	REQ-4, 5, 9, 13
Django Authentication Module	Registration, password verification, sessions/tokens, role enforcement.	/api/auth/*; permission checks.	User records; password hasher.	REQ-12, 13, 14
Django Booking Module	Availability search, create/cancel bookings, booking reference, staff management.	/api/tables/availability; /api/bookings.	Tables, bookings, users; DB transaction.	REQ-1 to REQ-4
Django Menu Module	Admin CRUD, customer listing of available items by category.	/api/menu endpoints.	Menu items; RBAC.	REQ-5 to REQ-7
Django Ordering Module	Validate cart, calculate total, create order/items, enforce status transition.	/api/orders endpoints.	Menu, orders, users; event publisher.	REQ-8, 9, 11
Node.js Real Time Service	Authenticate WebSocket connection; subscribe customer to own order room; emit status changes.	WSS Socket.io events.	Django internal event; current order status.	REQ-10
PostgreSQL Database	Store durable data; enforce keys, checks and transaction isolation.	Relational data service.	Django ORM; restricted Node.js read access.	All data-related requirements

3.2 Component Interaction Rules

- Only Django owns booking, menu, order, authentication, and authorization decisions. The Node.js service never writes business data directly.
- Django sends a real-time event only after the order update transaction commits successfully. This prevents a customer from seeing a state that was not saved.
- Node.js authorizes an order subscription by verifying the user identity and confirming that the customer owns the order or is authorized staff/admin.
- Both customer and staff views use REST as the recovery path after reconnecting, refreshing, or receiving an out-of-order event.

4 Security Architecture

4.1 Security Objectives

Objective	Design Response
Confidentiality	Use HTTPS and WSS; do not expose password hashes, internal database identifiers, or another customer's booking/order data.
Integrity	Validate all state changes server-side; use transactions; enforce allowed status transitions; record audit fields such as created_at, updated_at, and actor ID.
Availability	Return clear retry-safe errors, retain authoritative state in PostgreSQL, and let clients recover status through REST after WebSocket interruption.
Account protection	Use Django-supported salted password hashers, rate-limit login attempts, expire inactive sessions, and avoid detailed login-error disclosures.

4.2 Authentication and Session Management

The system uses Django authentication with an email or phone number and a password. Passwords are never stored or logged in plain text. On successful login, the server issues a secure session or signed token with a configured inactivity timeout of 30 minutes for this project. Each protected request revalidates the identity and role. A logout or inactivity expiration invalidates the session/token and returns 401 Unauthorized for further protected requests.

4.3 Authorization Matrix

Operation	Customer	Staff	Admin
Browse available menu	Allowed	Allowed	Allowed
Create own booking	Allowed	Allowed on behalf of customer	Allowed on behalf of customer
View bookings	Own bookings only	All current bookings	All bookings
Cancel booking	Own booking if at least 30 min remain	Manual cancel with reason	Manual cancel with reason
Create or edit menu item	Denied	Denied	Allowed
Place order / view own order	Allowed	May place on behalf of customer	Allowed
Update order status	Denied	Allowed	Allowed
Manage staff accounts	Denied	Denied	Allowed

4.4 Security Controls by Boundary

Boundary	Control
Browser to Django	HTTPS, CSRF protection for cookie sessions, CORS allowlist, server-side request validation, response fields limited by role.
Browser to Node.js	WSS, access credential at connection, per-order room authorization, reconnect requires re-authentication.
Django to PostgreSQL	ORM parameterization, least-privilege database account, transactional writes, backups for demo data.
Django to Node.js	Internal authenticated endpoint or signed event payload; Node.js accepts events only from the application service.
Admin actions	Role check, validation, audit fields, and user-visible error without exposing internal exception details.

4.5 Practical Threat Handling

Threat	Mitigation
SQL injection	Use Django ORM and parameterized queries; reject unvalidated filters; no string-built SQL.
Cross-site scripting	Escape template output, validate input, apply Django security middleware, avoid rendering raw user input.
Privilege escalation	Enforce permissions in every API view; hide buttons only as a usability feature, not as security.
Double booking race	Perform overlap check and booking creation inside one transaction with table-row locking or serializable protection.
Stale cart manipulation	Re-read each item availability and current price at checkout; calculate total on server.
Event disclosure	Authorize room subscription against the requesting user and order ownership before emitting status.

5 Detailed Software Design

5.1 Booking Design

A booking request includes date, start time, party size, and selected table. The server derives end time from the restaurant turnaround period, configured as 90 minutes in the initial release. The Booking module checks table capacity, locks the target table row, rechecks overlapping confirmed bookings, creates a confirmed booking, and generates a human-readable reference such as BK-1001. The database transaction commits before confirmation is returned. A cancellation is allowed only when server time is at least 30 minutes before the reservation start; it changes the booking status to Cancelled rather than deleting the record.

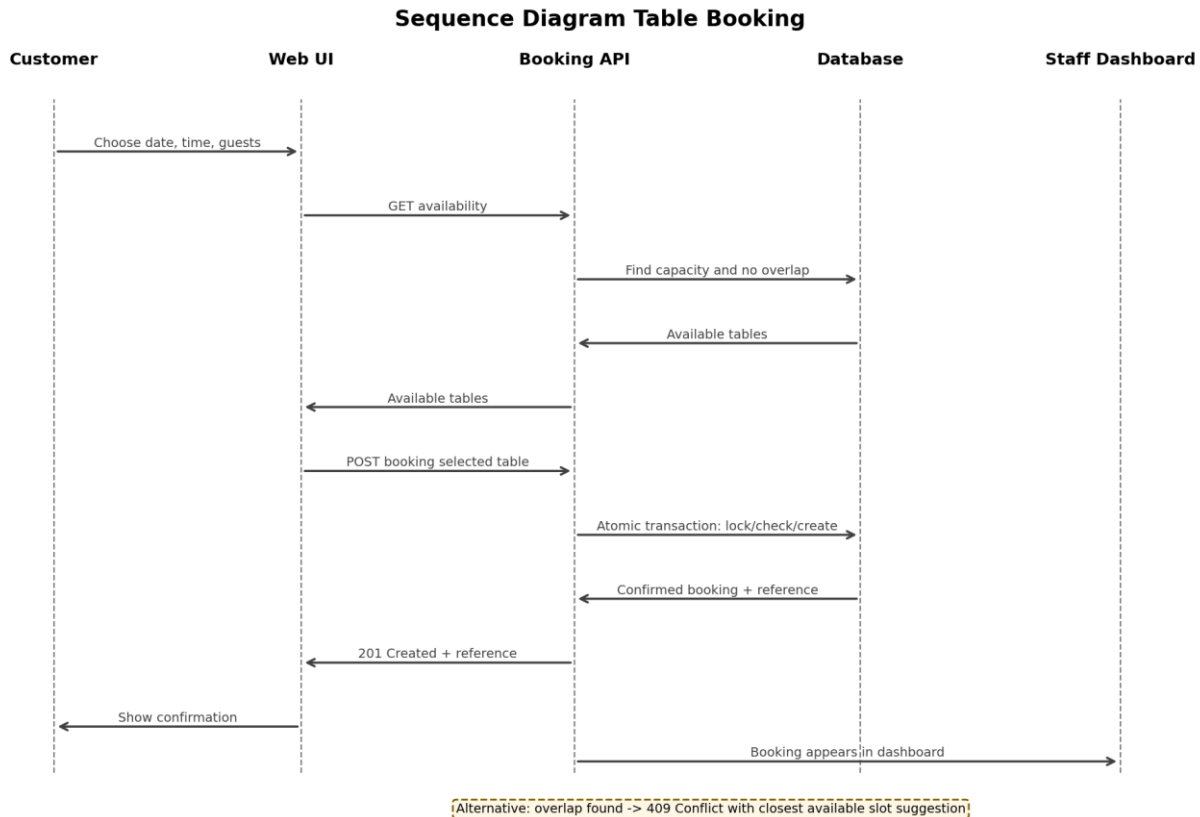


Figure 2. Sequence Diagram Table Booking

5.2 Food Ordering and Real Time Tracking

The cart exists in the browser until checkout. At checkout, the Ordering module treats the submitted items as untrusted: it loads every menu item again, checks that each is available, validates quantity, snapshots each accepted unit price into OrderItem, calculates the total on the server, and writes the order and items atomically. The initial status is Received. Staff may transition Received to Preparing and Preparing to Served. A cancelled transition is reserved for an approved exception before preparation and must record a reason.

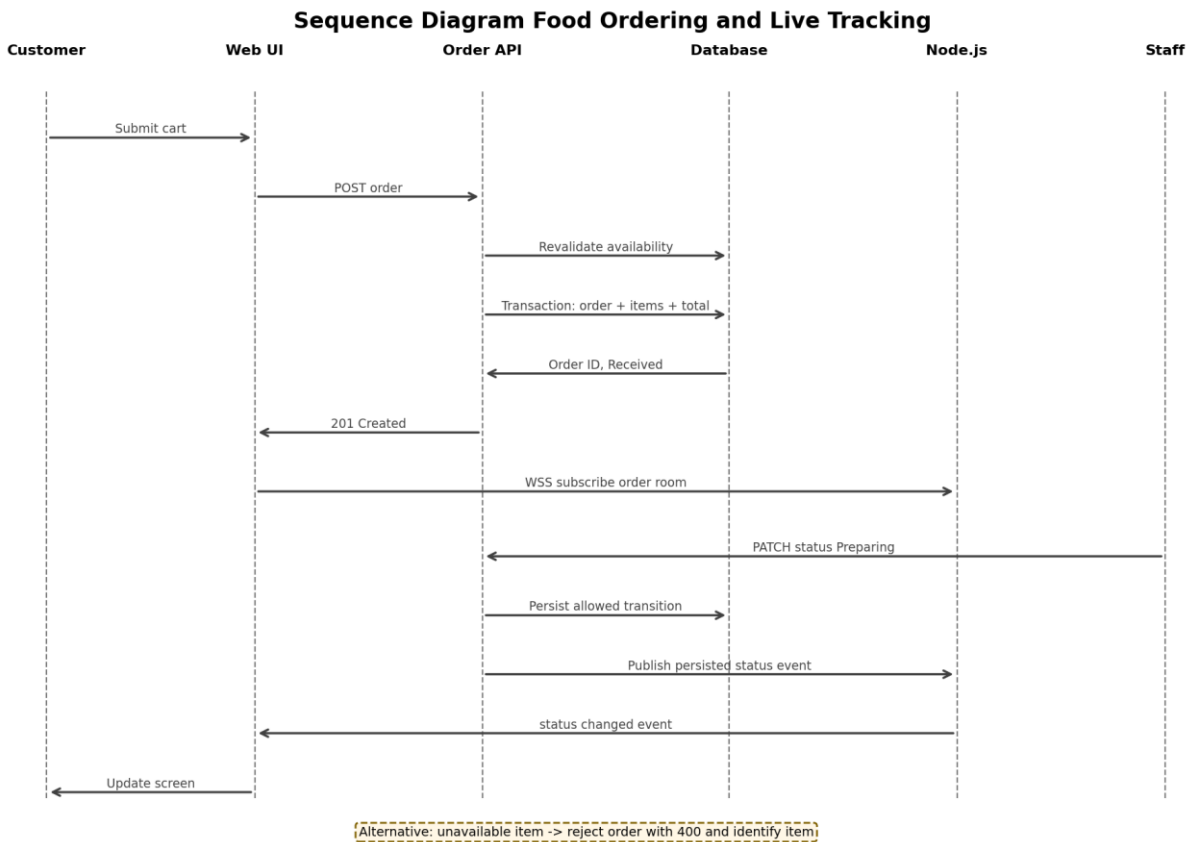


Figure 3. Sequence Diagram Food Ordering and Live Tracking

5 Detailed Software Design Continued

5.3 Authentication and Authorization

Registration checks uniqueness of email/phone and password policy. Login uses Django password verification, not a direct database comparison. Protected endpoints enforce authentication first and authorization second. The API uses 401 when the user has no valid authentication and 403 when the authenticated role lacks permission. These distinctions make errors actionable while avoiding disclosure of account existence.

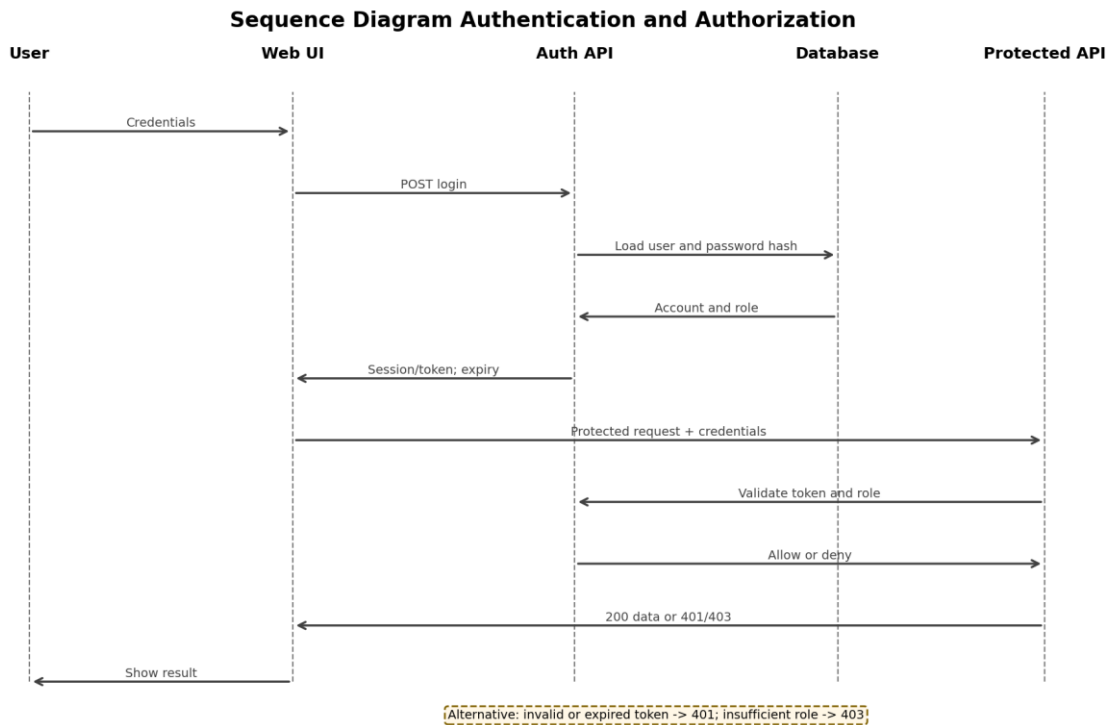


Figure 4. Sequence Diagram Authentication and Authorization

5.4 Order State Model

Order Lifecycle and Allowed Transitions



Each successful transition is persisted before the real-time event is emitted.

Figure 5. Order Lifecycle and Allowed Transitions

5.5 Idempotency and Recovery

Situation	Design Behavior
Customer resubmits booking after network uncertainty	The client supplies an idempotency key. Django returns the original confirmed booking for a repeated key instead of creating a duplicate.
Customer loses connection after placing order	The order is still committed if the server responded internally. On return, the customer uses GET /api/orders/{id} or the order history to retrieve the authoritative state.

Situation	Design Behavior
WebSocket disconnects	Client reconnects with valid authentication, re-subscribes to its order room, and requests current order status using REST.
Staff double-clicks status update	The API validates the current state; a repeated update to the same state is treated as a no-op or returns the current state, never creates a duplicate event.

6 Data Design

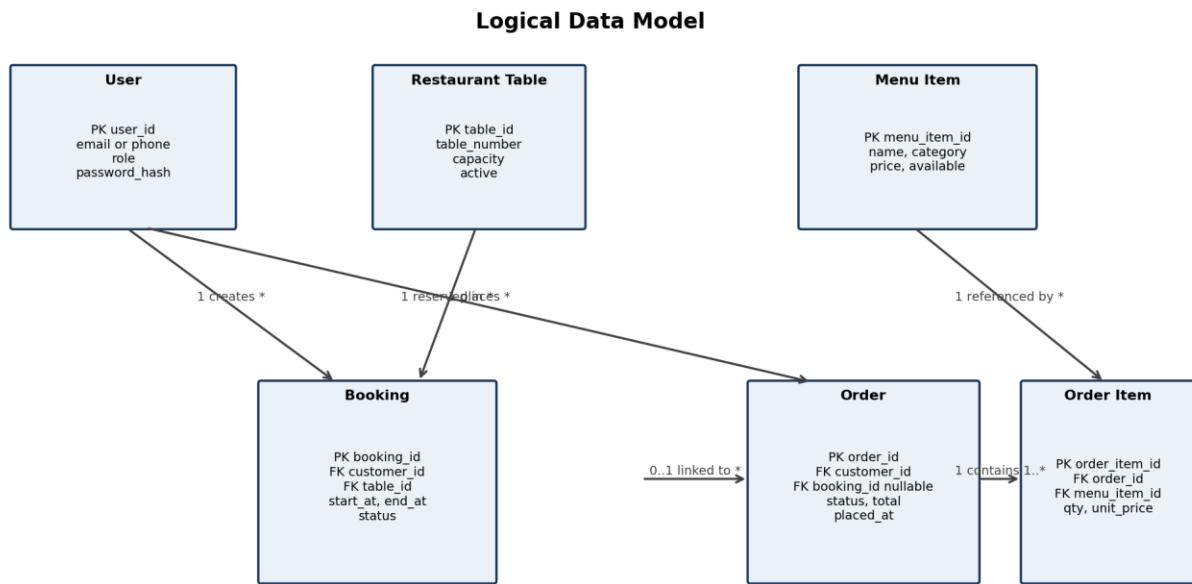


Figure 6. Logical Data Model

6.1 Core Tables and Constraints

Table	Key Fields	Important Constraints
User	user_id, email, phone, role, password_hash	Unique non-null login identifier; role in Customer, Staff, Admin; password_hash never returned by API.
RestaurantTable	table_id, table_number, capacity, active	Unique table_number; capacity greater than zero; inactive table cannot accept new booking.
Booking	booking_id, customer_id, table_id, start_at, end_at, status	start_at before end_at; party size greater than zero; Cancelled records retained; overlap prevention enforced in transaction.
MenuItem	menu_item_id, name, category, price, available	Price non-negative; category required; unavailable items excluded from customer listing.
Order	order_id, customer_id, booking_id, status, total_amount, placed_at	Status constrained to valid lifecycle; total non-negative; booking reference optional.
OrderItem	order_item_id, order_id, menu_item_id, quantity, unit_price	Quantity greater than zero; unit_price non-negative; unit_price is a snapshot at checkout.

6.2 Double Booking Prevention

Availability shown to the user is advisory because another customer can act at the same time. The decisive check happens during booking creation. Django begins a transaction, locks the selected table row, checks

whether a non-cancelled booking exists for the same table where existing.start_at is before requested.end_at and existing.end_at is after requested.start_at, and creates the booking only when no overlap exists. If an overlap is found, the transaction is rolled back and the API returns 409 Conflict with alternative slots. For PostgreSQL, an exclusion constraint on the table/time range is a strong additional safeguard; for the mini-project, transaction plus row lock is the minimum implementation standard.

6.3 Data Retention and Audit Fields

Bookings and orders are not hard-deleted during normal operations. Each retains created_at, updated_at, and the acting user where relevant. Menu items referenced by open or historical orders should be marked unavailable rather than deleted; a hard delete is allowed only when the item has no order references. This preserves report accuracy and prevents a past order from losing its item name or price.

7 API and Interface Design

7.1 REST API Conventions

All REST endpoints use JSON over HTTPS and the /api prefix. Protected endpoints require a valid session/token. The server validates inputs, uses consistent HTTP status codes, and returns an error object with code, message, and optional field details. IDs in examples are illustrative and not a claim of existing implementation.

Method	Endpoint	Authorization	Purpose
POST	/api/auth/register	Public	Create a customer account.
POST	/api/auth/login	Public	Authenticate and issue session/token.
POST	/api/auth/logout	Authenticated	Invalidate active session/token.
GET	/api/tables/availability	Public or authenticated	Return tables that fit requested date, time, and guests.
POST	/api/bookings	Customer or staff/admin	Create a reservation using atomic availability validation.
GET	/api/bookings	Authenticated	Customer gets own records; staff/admin get permitted records with filters.
PATCH	/api/bookings/{id}/cancel	Owner or staff/admin	Cancel according to time and role rules.
GET	/api/menu	Public	Return available menu items grouped by category.
POST	/api/menu	Admin	Create menu item.
PATCH	/api/menu/{id}	Admin	Update menu item or availability.
DELETE	/api/menu/{id}	Admin	Delete only when referential rules permit.
POST	/api/orders	Customer or staff/admin	Validate cart and create an order.
GET	/api/orders/{id}	Owner or staff/admin	Return authoritative order and items.
PATCH	/api/orders/{id}/status	Staff or admin	Apply an allowed order-status transition.

7.2 WebSocket Events

Event	Direction	Payload	Authorization and Reliability
subscribe_order	Browser to Node.js	order_id; access credential	Node.js verifies identity and ownership/role before joining the room.
order_status_changed	Node.js to Browser	order_id, status, updated_at, version	Sent only after Django persistence; client compares version and may fetch current REST state.

Event	Direction	Payload	Authorization and Reliability
error	Node.js to Browser	code, message	Does not expose internal details; client retries only when appropriate.

7.3 User Interface Rules

- Booking form shows date, time, party size, available table options, final confirmation reference, and cancellation rule.
- Customer menu hides unavailable items and groups remaining items by category.
- Checkout clearly identifies items rejected after revalidation and preserves the rest of the cart for correction.
- Staff dashboard separates bookings, current orders, and admin-only menu management so role boundaries are visible.
- Every form displays validation messages next to the relevant field and retains non-sensitive entered values where safe.

8 Error Handling and Validation

8.1 Standard Error Response

Field	Meaning
code	Stable machine-readable error identifier, for example TABLE_UNAVAILABLE.
message	Short, user-safe explanation.
details	Optional map of field-specific validation messages or suggested alternate slots.
request_id	Optional server correlation ID for staff debugging; never includes secrets.

8.2 Error Catalog

Scenario	HTTP or Event Result	User-Facing Behavior	Server Action
Invalid or missing input	400 Bad Request	Highlight invalid field and explain correction.	Reject request before any write.
Unauthenticated protected request	401 Unauthorized	Ask the user to sign in again.	Do not reveal protected data.
Authenticated but unauthorized	403 Forbidden	Show permission message.	Log authorization denial at appropriate level.
Table slot conflict	409 Conflict	Explain that the table was just booked and suggest closest slot.	Rollback transaction; no booking created.
Duplicate registration identifier	409 Conflict	State that the identifier is already in use without exposing other account data.	Do not create user.
Unavailable cart item	400 Bad Request	Name unavailable item and return cart correction data.	Do not create partial order.
Invalid order transition	409 Conflict	Show current order state and permitted action.	Do not publish real-time event.
Resource not found	404 Not Found	Show not-found state.	Return no extra internal information.
Unexpected server error	500 Internal Server Error	Show a safe retry message and request ID.	Log stack trace internally; rollback transaction.
WebSocket interruption	Reconnect event or timeout	Show reconnecting indicator; refresh authoritative status after reconnect.	No business data is changed by reconnect.

8.3 Validation Rules

Input Area	Rules
------------	-------

Input Area	Rules
Booking	Future date/time; positive party size; active table capacity at least party size; no overlap; cancellation at least 30 minutes before start for customer.
Menu item	Required name and category; price zero or greater; availability is boolean.
Order	At least one item; positive integer quantity; all items available at checkout; total calculated only on server.
Account	Unique email or phone; password policy; role is server-controlled and customer registration creates Customer role only.
Order status	Only Received to Preparing to Served in normal flow; exception cancellation must be authorized and audited.

9 Deployment and Quality Attributes

Deployment View for a Single Restaurant Outlet

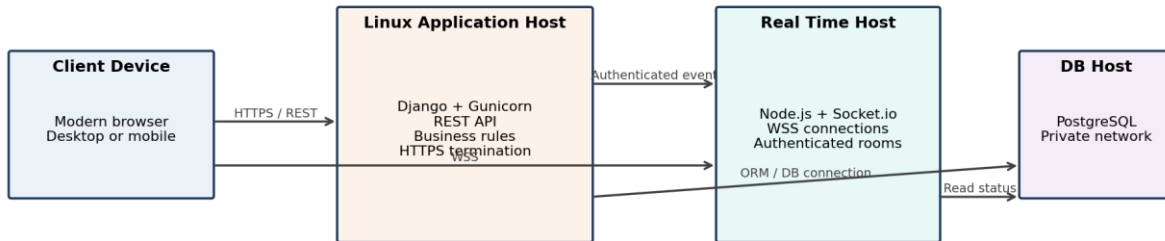


Figure 7. Deployment View for a Single Restaurant Outlet

9.1 Deployment Principles

The recommended demonstration deployment has Django and Node.js as separate processes behind an HTTPS-capable reverse proxy, with PostgreSQL on a private database connection. Environment secrets are stored outside source control. SQLite may be used only for a classroom demonstration with limited concurrent activity; PostgreSQL is the preferred choice when validating the 50-user and booking-concurrency targets.

9.2 Quality Attribute Tactics

Quality Attribute	SRS Target	Design Tactic	Verification Reference
Performance	Page/API response within 2 seconds under normal load	Index booking start time/table, paginate staff lists, avoid N+1 ORM queries, keep WebSocket payload small.	Add NFR performance test set before final submission.
Concurrency	At least 50 concurrent users	Stateless API requests, database transactions, connection pooling, and load test with realistic booking/order mix.	Add NFR concurrent-user test.
Real-time latency	Order status appears within 1 second	Persist then publish compact event; instrument publish and client-receive times.	IT_06 and IT_07, plus timing assertion.
Reliability	Confirmed booking/order data is not lost	Atomic database transactions, durable PostgreSQL storage, retry-safe request IDs, recovery through GET APIs.	IT_01 and IT_08.
Usability	No training for customer flows	Clear labels, inline validation, few-step booking/order flow, status labels.	ST_01, ST_05, and usability walkthrough.
Maintainability	Separate feature modules	Django apps for accounts, booking, menu, orders; documented API and interfaces.	Code review against SAD.
Portability	Latest major browsers	Responsive HTML/CSS; browser smoke tests on Chrome, Firefox, and Edge.	Add browser compatibility test.

9.3 Monitoring for a Real Restaurant

For practical operation, the system should log booking conflicts, order-validation rejections, failed login attempts, status-transition errors, and WebSocket delivery failures with request IDs and timestamps. These records let staff distinguish a genuine booking conflict from a device or connectivity problem. Logs must not contain passwords, tokens, or complete sensitive personal data.

10 Requirements Traceability and Verification

This matrix connects every approved functional requirement to the responsible design area and existing test plan IDs. The design does not claim that these tests have already passed; it defines how the implementation will be verified.

Requirement	Design Element	Primary Verification
REQ-1	Booking module; availability query; Figure 2; transaction design	UT_01, UT_02, IT_01, ST_01, ST_02
REQ-2	Booking transaction, row lock, overlap check; Section 6.2	UT_03, UT_04, IT_01, ST_02
REQ-3	Cancellation endpoint and time-bound validation	UT_05
REQ-4	Staff/Admin booking dashboard and protected booking APIs	IT_02, IT_03
REQ-5	Admin-only menu CRUD; RBAC and menu data rules	UT_06 to UT_09, IT_04, ST_03
REQ-6	Customer menu availability filter	UT_10, IT_04, IT_05, ST_04
REQ-7	Category field and grouped menu response/UI	UT_11, ST_03
REQ-8	Cart revalidation; Order and OrderItem transaction; Figure 3	UT_12, UT_13, UT_16, IT_08, ST_05
REQ-9	Order state model; protected status endpoint; Figure 5	UT_13, UT_14, ST_05
REQ-10	Django committed event to Node.js; authenticated order room	IT_06, IT_07, ST_05
REQ-11	Checkout availability revalidation and error response	UT_15, ST_06
REQ-12	Registration/login; password hasher; Figure 4	UT_17, UT_18, UT_19, IT_10, ST_07
REQ-13	RBAC matrix and protected endpoints; Figure 4	IT_09, IT_10, ST_08
REQ-14	Inactivity expiry and reauthentication handling	UT_20

10.1 Nonfunctional Verification Gaps to Add Before Final Submission

The supplied test plan has strong functional coverage, but it should be updated with explicit nonfunctional test cases before final submission. These tests strengthen the claim that the project works beyond ideal flows.

Suggested ID	Test Objective	Pass Criterion
NFT_01	Measure availability, booking, menu, order, and status APIs under normal data volume.	95 percent of measured responses complete within 2 seconds under normal load.
NFT_02	Run a load scenario with at least 50 concurrent users doing browse, availability, and order operations.	No data corruption; error rate within accepted project threshold; system remains responsive.

Suggested ID	Test Objective	Pass Criterion
NFT_03	Measure real-time update delay after staff status change.	Customer receives persisted status event within 1 second in the test environment.
NFT_04	Attempt two concurrent reservations of the same table/time.	Exactly one booking succeeds; the other receives 409 Conflict.
NFT_05	Security validation of protected endpoints, expired sessions, and role boundaries.	Unauthorized requests fail with 401 or 403 and reveal no protected data.
NFT_06	Browser compatibility smoke test.	Core customer and staff workflows work on latest Chrome, Firefox, and Edge.

Appendix A Glossary and Assumptions

Term	Definition
RMS	Restaurant Management System.
Admin	Most privileged role; manages menu and staff-facing operations.
Staff	Restaurant employee who views bookings and updates orders.
Customer	User who books a table and/or places an order.
Booking interval	Reservation start time through computed end time; default turnaround is 90 minutes.
REST API	HTTPS JSON interface exposed by Django REST Framework.
WebSocket	Persistent WSS connection used for server-pushed order-status updates.
RBAC	Role-based access control.

Assumptions

- The project represents a single restaurant outlet and operates in English.
- Table capacity and active status are managed by the restaurant.
- A booking corresponds to one table and the standard turnaround duration is configurable.
- Payment processing is outside Version 1.0 scope.
- The initial implementation may use Django Admin to accelerate trusted admin-side CRUD, but the same authorization and audit rules apply.

Appendix B Representative API Payloads

B.1 Create Booking Request

```
{
  "table_id": 5,
  "start_at": "2026-10-15T19:00:00+05:30",
  "party_size": 2,
  "idempotency_key": "b7e0..."
}
```

B.2 Successful Booking Response

```
{
  "booking_id": "BK-1001",
  "status": "Confirmed",
  "start_at": "2026-10-15T19:00:00+05:30",
  "end_at": "2026-10-15T20:30:00+05:30"
}
```

B.3 Table Conflict Response

```
{
  "code": "TABLE_UNAVAILABLE",
  "message": "The selected table is no longer available for that time.",
  "details": {"suggested_start_at": "2026-10-15T20:30:00+05:30"}
}
```

B.4 Order Status WebSocket Event

```
{
  "event": "order_status_changed",
  "order_id": "OR-2001",
  "status": "Preparing",
  "updated_at": "2026-10-15T19:18:12+05:30",
  "version": 2
}
```